

LABORATORIO 01

La tabla que recuerda

Guía para leer mientras trabajas el cuaderno. Cada celda trae la pregunta que responde, la idea en el almacén, la sentencia explicada parte por parte, lo que sale en pantalla y cómo leerlo. Las salidas son las de la solución ejecutada del repositorio.

Introducción

1 · El tema

En este laboratorio corriges un dato y después preguntas qué decía antes. Araucaria Ferretería pasa de PEQUENA a MEDIANA, y la tabla, además de guardar el valor nuevo, se acuerda del viejo. Vas a mirar el pasado de dos formas y después vas a volver a él de verdad.

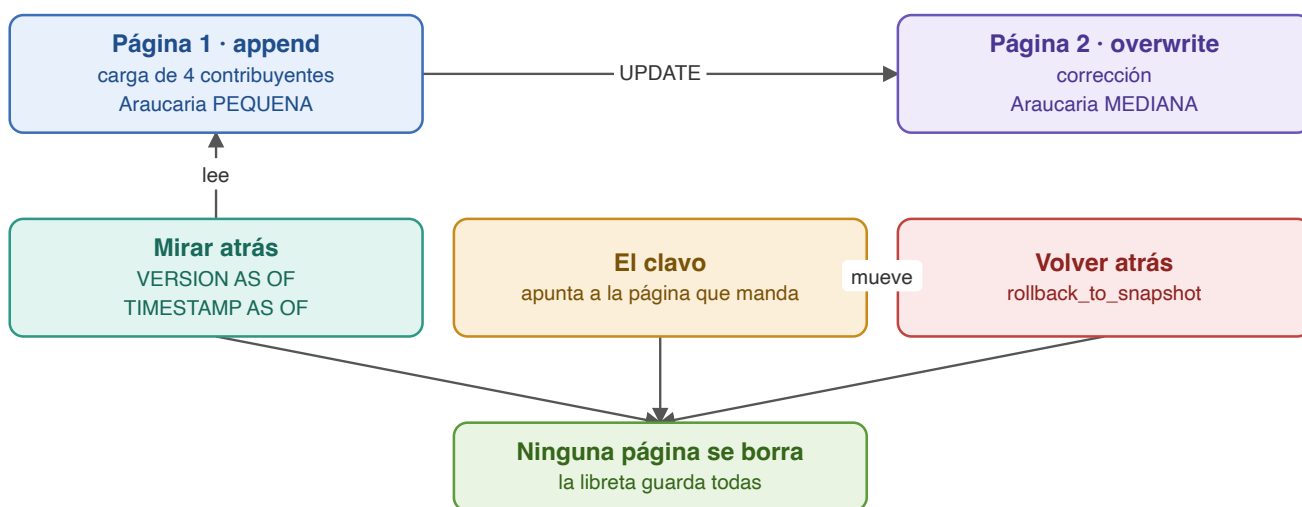
2 · El problema, en el almacén

El almacenero corrige un renglón de su libreta. Si lo corrige con goma, el valor anterior desaparece. Cuando un inspector llega preguntando qué decía ese renglón el mes pasado, no hay cómo responder. Y si la corrección estaba mal, tampoco hay cómo deshacerla, porque lo que había se borró.

3 · El problema en términos técnicos

En una tabla común, un `UPDATE` reemplaza el valor y el anterior se pierde. Para saber qué decía un registro en una fecha hay que tener respaldos, restaurarlos en otra parte y buscar ahí, lo que toma horas. Para deshacer una carga mala pasa lo mismo, hay que restaurar desde un respaldo.

4 · El diagrama



Página 1 · append (azul). Es la carga de los cuatro contribuyentes. Ahí Araucaria dice PEQUENA.

Página 2 · overwrite (morado). Es la corrección. Iceberg no borra la página 1, escribe una página nueva donde Araucaria dice MEDIANA, y esa pasa a mandar.

Mirar atrás (verde agua). Es leer una página anterior, por su número o por su fecha. La libreta no cambia y sigue mandando la última página.

El clavo (amarillo). Es el catálogo. Dice cuál página manda ahora.

Volver atrás (rojo). Es declarar que una página anterior vuelve a mandar. Esto sí cambia el clavo.

Ninguna página se borra (verde). Ni mirar ni volver borran nada. Las páginas siguen en la libreta.

5 · La solución

En el almacén, el almacenero nunca usa goma. Cada vez que corrige, escribe una página nueva con la fecha y deja la anterior en la libreta. Para saber qué decía un renglón el mes pasado, hojea hacia atrás. Si la corrección estaba mal, declara que la página de antes vuelve a ser la que manda.

En Iceberg cada escritura deja una página nueva, que se llama snapshot. `VERSION AS OF` y `TIMESTAMP AS OF` leen una página anterior sin cambiar nada. `rollback_to_snapshot` declara vigente una página anterior. Mirar atrás y volver atrás no son lo mismo, y esa diferencia es todo el laboratorio.

6 · Los pasos

- **Paso 0.** Creas la tabla y cargas cuatro contribuyentes.
- **Paso 1.** Corriges el segmento de Araucaria.
- **Paso 2.** Ves que la tabla guardó dos páginas.
- **Paso 3.** Miras el pasado por número de página y por fecha, sin cambiar nada.
- **Paso 4.** Vuelves al pasado de verdad y miras el historial.

Si quieres repetir el laboratorio desde el principio, corre este comando desde la carpeta del repositorio.

```
bash
bin/reiniciar-lab.sh 01
```

Paso 0 · Preparar la tabla

Celda 0.1 · Ponerte en tu espacio

La pregunta. ¿En qué espacio vas a trabajar?

Por qué ahora. Todo lo que crees tiene que quedar en tu espacio, `mi_espacio`.

En el almacén. Es ponerte frente a tu estante antes de abrir una libreta.

La sentencia, parte por parte.

- `%%sql` es la primera línea de toda celda SQL del cuaderno. Le dice al cuaderno que lo que sigue es SQL y no Python.
- `-- Celda 0.1` es un comentario. El motor ignora lo que va después de dos guiones.
- `USE mi_espacio` cambia el espacio activo. Desde aquí, las tablas sin espacio adelante se buscan en `mi_espacio`.

```
sql
%%sql
-- Celda 0.1
USE mi_espacio
```

Lo que sale en pantalla.

```
text
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/09/25 18:21:47 WARN NativeCodeLoader: Unable to load native-hadoop library for your
platform... using builtin-java classes where applicable
Listo. La sentencia se ejecutó.
```

Cómo se lee. Quedaste en `mi_espacio`. La última línea es el aviso del cuaderno de que la sentencia terminó bien.

Las tres primeras líneas no son errores. Salen solo en la primera celda, cuando arranca Spark.

- `Setting default log level to "WARN"` dice que Spark va a mostrar solo los avisos y los errores.
- `To adjust logging level use sc.setLogLevel(newLevel)` explica cómo cambiar eso. `sc` es el contexto de Spark y `setLogLevel` la función que cambia el nivel. SparkR es la versión de Spark para el lenguaje R, que aquí no se usa.
- `WARN NativeCodeLoader: Unable to load native-hadoop library` dice que Spark no encontró una biblioteca de Hadoop escrita para este sistema operativo y usa la versión en Java. Funciona igual. `26/09/25 18:21:47` es la fecha y la hora del aviso, en UTC.

Celda 0.2 · Partir de cero

La pregunta. ¿Cómo te aseguras de partir con la tabla vacía?

Por qué ahora. Si ya corriste el laboratorio, la tabla existe y el `CREATE TABLE` fallaría.

En el almacén. Es sacar del estante la libreta vieja con ese nombre, si la hay.

La sentencia, parte por parte.

- `DROP TABLE` borra una tabla.
- `IF EXISTS` hace que no se queje si la tabla no existe.
- `contribuyentes_lab01` es la tabla del laboratorio.

```
sql
%%sql
-- Celda 0.2
DROP TABLE IF EXISTS contribuyentes_lab01
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. Terminó bien. Si la tabla no existía, no hizo nada.

Celda 0.3 · Crear la tabla

La pregunta. ¿Cómo es la tabla de hoy?

Por qué ahora. Hace falta una tabla Iceberg con tres columnas para el padrón.

En el almacén. Es abrir una libreta nueva y dibujar tres columnas.

La sentencia, parte por parte.

- `CREATE TABLE contribuyentes_lab01` crea la tabla en tu espacio.
- `rut STRING`, `razon_social STRING` y `segmento STRING` son tres columnas de texto.
- `USING iceberg` hace que sea una tabla Iceberg, que guarda cada escritura como una página nueva.

```
sql
%%sql
-- Celda 0.3
CREATE TABLE contribuyentes_lab01 (
  rut          STRING,
  razon_social STRING,
  segmento     STRING
) USING iceberg
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. La tabla quedó creada y vacía.

Celda 0.4 · Cargar cuatro contribuyentes

La pregunta. ¿Qué datos tiene la tabla al partir?

Por qué ahora. Esta carga deja la primera página de la libreta, la que después vas a mirar y a la que vas a volver.

En el almacén. Es anotar cuatro renglones. Queda la página 1.

La sentencia, parte por parte.

- `INSERT INTO contribuyentes_lab01` agrega filas a la tabla.
- `VALUES` introduce las filas escritas a mano, cada una entre paréntesis y separadas por comas.
- Cada fila trae RUT, razón social y segmento, en el orden de las columnas. Son un contribuyente por segmento.

```
sql
%%sql
-- Celda 0.4
INSERT INTO contribuyentes_lab01 VALUES
  ('77746521-K', 'Pehuen Logistica EIRL', 'MICRO'),
  ('77884562-8', 'Araucaria Ferreteria SpA', 'PEQUENA'),
  ('78800840-6', 'Copihue Maquinarias EIRL', 'MEDIANA'),
  ('76171162-8', 'Huemul Alimentos EIRL', 'GRANDE')
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. Las cuatro filas quedaron escritas.

Celda 0.5 · Mirar lo cargado

La pregunta. ¿Quedaron bien las cuatro filas?

Por qué ahora. Antes de corregir, conviene ver cómo estaba todo.

En el almacén. Es leer la página 1 completa.

La sentencia, parte por parte.

- `SELECT *` pide todas las columnas.
- `FROM contribuyentes_lab01` es la tabla.
- `ORDER BY rut` ordena por RUT. Sin él, el orden de las filas no está garantizado y podría cambiar de una vez a otra.

```
sql
%%sql
-- Celda 0.5
SELECT * FROM contribuyentes_lab01 ORDER BY rut
```

Lo que sale en pantalla.

```
text
rut      razon_social      segmento
76171162-8  Huemul Alimentos EIRL  GRANDE
77746521-K  Pehuen Logistica EIRL  MICRO
77884562-8  Araucaria Ferreteria SpA  PEQUENA
78800840-6  Copihue Maquinarias EIRL  MEDIANA
4 filas.
```

Cómo se lee. Cuatro contribuyentes. Araucaria Ferretería, RUT 77884562-8 , está en PEQUENA. Es la fila que vas a corregir.

Paso 1 · Corregir un dato

Celda 1.1 · La corrección

La pregunta. ¿Cómo se cambia el segmento de un contribuyente?

Por qué ahora. Araucaria creció y pasa de PEQUENA a MEDIANA. Es el trámite más común de una administración tributaria.

En el almacén. Es corregir el renglón de Araucaria. En esta libreta no hay goma, así que queda una página nueva.

La sentencia, parte por parte.

- `UPDATE contribuyentes_lab01` cambia filas que ya existen.
- `SET segmento = 'MEDIANA'` dice qué columna cambia y a qué valor.
- `WHERE rut = '77884562-8'` dice a qué filas. Solo a la de Araucaria.

```
sql
%%sql
-- Celda 1.1
UPDATE contribuyentes_lab01 SET segmento = 'MEDIANA' WHERE rut = '77884562-8'
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. La corrección se hizo.

Celda 1.2 · Ver la tabla corregida

La pregunta. ¿Qué dice la tabla ahora?

Por qué ahora. Para comprobar que la corrección quedó.

En el almacén. Es leer la libreta como está hoy.

La sentencia, parte por parte.

- `SELECT * FROM contribuyentes_lab01` lee toda la tabla.
- `ORDER BY rut` ordena por RUT.

```
sql
%%sql
-- Celda 1.2
SELECT * FROM contribuyentes_lab01 ORDER BY rut
```

Lo que sale en pantalla.

```
text
rut      razon_social      segmento
76171162-8  Huemul Alimentos EIRL  GRANDE
77746521-K  Pehuen Logistica EIRL  MICRO
77884562-8  Araucaria Ferreteria SpA MEDIANA
78800840-6  Copihue Maquinarias EIRL MEDIANA
4 filas.
```

Cómo se lee. Araucaria dice MEDIANA. PEQUENA desapareció de la vista. Hasta aquí es lo mismo que haría cualquier base de datos.

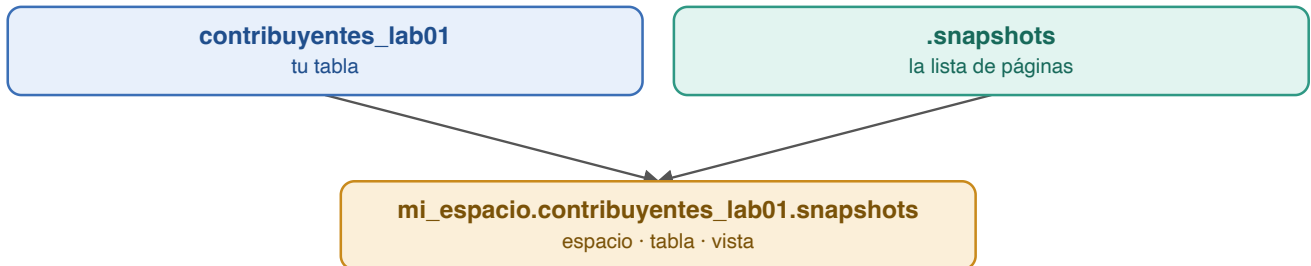
Paso 2 · Descubrir que hay historia

Celda 2.1 · La lista de páginas

La pregunta. ¿Qué páginas tiene la libreta?

Por qué ahora. Si la tabla guardó la versión anterior, tiene que haber más de una página.

En el almacén. Es mirar la lista de páginas de la libreta, con su fecha.



contribuyentes_lab01 (azul). Es tu tabla, con sus datos.

.snapshots (verde agua). Es una vista de sistema que cuelga de la tabla. No tiene datos del padrón, tiene la lista de páginas.

mi_espacio.contribuyentes_lab01.snapshots (amarillo). Para llegar a la vista hay que nombrar las tres partes, aunque ya estés en tu espacio. Si escribes solo `contribuyentes_lab01.snapshots`, SQL cree que `contribuyentes_lab01` es un espacio y dice que no lo encuentra.

La sentencia, parte por parte.

- `SELECT snapshot_id, committed_at, operation` pide tres columnas. `snapshot_id` es el número de la página, `committed_at` la hora en que se escribió, en UTC, y `operation` qué se hizo.
- `FROM mi_espacio.contribuyentes_lab01.snapshots` es la vista de sistema, con sus tres partes.
- `ORDER BY committed_at` ordena de la página más vieja a la más nueva.

```
sql
%%sql
-- Celda 2.1
SELECT snapshot_id, committed_at, operation
FROM mi_espacio.contribuyentes_lab01.snapshots
ORDER BY committed_at
```

Lo que sale en pantalla.

```
text
snapshot_id      committed_at      operation
3093317371693715250  2026-09-25 18:21:54.339  append
962689764919215466  2026-09-25 18:21:57.535  overwrite
2 filas.
```

Cómo se lee. Dos páginas.

- La primera, `3093317371693715250`, de las 18:21:54.339 UTC, las 15:21 en Chile, es `append`, que significa que se agregaron filas. Es la carga de los cuatro contribuyentes.
- La segunda, `962689764919215466`, de las 18:21:57.535 UTC, es `overwrite`, que significa que se reemplazaron filas. Es la corrección de Araucaria.

La tabla no tiene ninguna columna con versiones. La memoria está en estas páginas.

Paso 3 · Mirar el pasado

Celda 3.1 · La tabla en la primera página

La pregunta. ¿Cómo estaba la tabla en la primera página?

Por qué ahora. Ya sabes que la página 1 existe. Ahora la lees.

En el almacén. Es hojear hacia atrás hasta la página 1 y leerla, sin tocar la libreta.

La sentencia, parte por parte.

- `SELECT * FROM contribuyentes_lab01` lee la tabla.
- `VERSION AS OF <TU_SNAPSHOT_ID>` pide la tabla como estaba en esa página. Donde dice `<TU_SNAPSHOT_ID>` escribes el `snapshot_id` de la primera fila de la celda 2.1, sin comillas. En la solución fue `3093317371693715250`. El tuyo es otro, porque cada tabla genera sus propios números.
- `ORDER BY rut` ordena por RUT.

```
sql
%%sql
-- Celda 3.1
SELECT * FROM contribuyentes_lab01 VERSION AS OF <TU_SNAPSHOT_ID> ORDER BY rut
```

Lo que sale en pantalla.

```
text
rut          razon_social          segmento
76171162-8   Huemul Alimentos EIRL          GRANDE
77746521-K   Pehuen Logistica EIRL          MICRO
77884562-8   Araucaria Ferreteria SpA       PEQUENA
78800840-6   Copihue Maquinarias EIRL       MEDIANA
4 filas.
```

Cómo se lee. Araucaria dice PEQUENA otra vez. Es la tabla como estaba antes de la corrección.

Celda 3.2 · La tabla de hoy, otra vez

La pregunta. ¿Mirar atrás cambió algo?

Por qué ahora. Hay que comprobar que leer una página vieja no cambió la tabla.

En el almacén. Es cerrar la libreta y volver a abrirla en la última página.

La sentencia, parte por parte.

- `SELECT * FROM contribuyentes_lab01 ORDER BY rut` es la consulta normal, sin pedir ninguna versión.

```
sql
%%sql
-- Celda 3.2
SELECT * FROM contribuyentes_lab01 ORDER BY rut
```

Lo que sale en pantalla.

```
text
rut      razon_social      segmento
76171162-8  Huemul Alimentos EIRL    GRANDE
77746521-K  Pehuen Logistica EIRL    MICRO
77884562-8  Araucaria Ferreteria SpA MEDIANA
78800840-6  Copihue Maquinarias EIRL MEDIANA
4 filas.
```

Cómo se lee. Sigue diciendo MEDIANA. Mirar atrás no cambió nada. Sigue mandando la última página, y esto se puede hacer en producción sin riesgo.

Celda 3.3 · Mirar el pasado por fecha

La pregunta. ¿Se puede pedir la tabla por fecha y hora en vez de por número?

Por qué ahora. Nadie llega preguntando por un número de página. Un auditor pregunta cómo estaba el dato en una fecha.

En el almacén. Es buscar en la libreta la página que mandaba a cierta hora.

La sentencia, parte por parte.

- `TIMESTAMP AS OF '<TU_COMMITTED_AT>'` pide la tabla como estaba en ese instante. Devuelve la última página escrita hasta esa hora. Donde dice `<TU_COMMITTED_AT>` escribes, entre comillas, el `committed_at` de la primera fila de la celda 2.1, tal como sale. En la solución fue `2026-09-25 18:21:54.339`.
- `SELECT * FROM contribuyentes_lab01` y `ORDER BY rut` son como antes.

```
sql
%%sql
-- Celda 3.3
SELECT * FROM contribuyentes_lab01 TIMESTAMP AS OF '<TU_COMMITTED_AT>' ORDER BY rut
```

Lo que sale en pantalla.

```
text
rut      razon_social      segmento
76171162-8  Huemul Alimentos EIRL    GRANDE
77746521-K  Pehuen Logistica EIRL    MICRO
77884562-8  Araucaria Ferreteria SpA PEQUENA
78800840-6  Copihue Maquinarias EIRL MEDIANA
4 filas.
```

Cómo se lee. Araucaria dice PEQUENA. El mismo resultado de la celda 3.1, preguntando por tiempo en vez de por número. La hora se interpreta en UTC, la misma en que la muestra `.snapshots`.

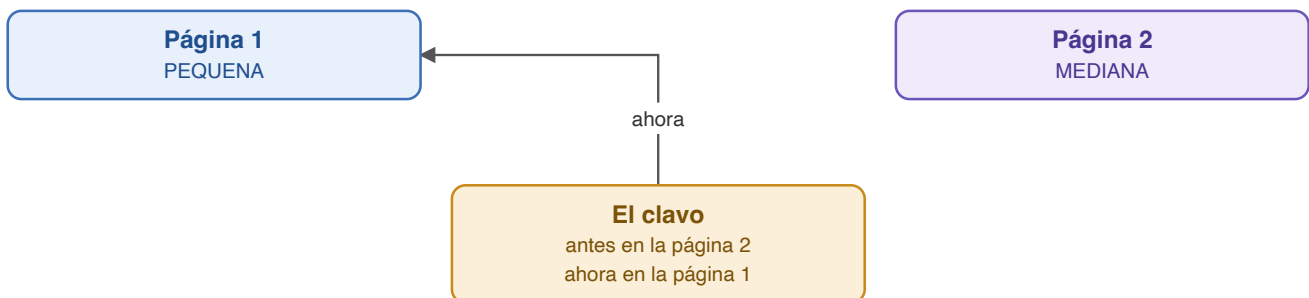
Paso 4 · Volver atrás de verdad

Celda 4.1 · Declarar vigente la primera página

La pregunta. ¿Cómo se deshace la corrección?

Por qué ahora. Hasta aquí solo leíste el pasado. El caso real es un proceso que cargó mal y hay que deshacerlo.

En el almacén. Es mover el clavo para que la libreta vuelva a abrirse en la página 1. La página 2 no se arranca, queda en la libreta.



Página 1 (azul). Vuelve a mandar. Lo que se lea desde ahora sale de aquí.

Página 2 (morado). Sigue en la libreta, pero deja de mandar.

El clavo (amarillo). `rollback_to_snapshot` solo cambia a qué página apunta el clavo.

La sentencia, parte por parte.

- `CALL` ejecuta un procedimiento, que es una operación del sistema.
- `spark_catalog.system.rollback_to_snapshot` es el procedimiento de Iceberg que vuelve a una página. `spark_catalog` es el catálogo y `system` el grupo de procedimientos.
- `'mi_espacio.contribuyentes_lab01'` es la tabla, entre comillas y con su espacio adelante.
- `<TU_SNAPSHOT_ID>` es el mismo número de la celda 3.1, sin comillas. En la solución fue `3093317371693715250`.

```
sql
%%sql
-- Celda 4.1
CALL spark_catalog.system.rollback_to_snapshot('mi_espacio.contribuyentes_lab01',
<TU_SNAPSHOT_ID>)
```

Lo que sale en pantalla.

```
text
previous_snapshot_id  current_snapshot_id
962689764919215466   3093317371693715250
1 fila.
```

Cómo se lee. Dos números. `previous_snapshot_id` es la página que mandaba antes, `962689764919215466`, la de la corrección. `current_snapshot_id` es la que manda ahora, `3093317371693715250`, la de la carga.

Celda 4.2 · La tabla después de volver

La pregunta. ¿Qué dice la tabla ahora, sin pedir ninguna versión?

Por qué ahora. Para ver que la vuelta atrás sí cambió la tabla.

En el almacén. Es abrir la libreta de nuevo, que ahora se abre en la página 1.

La sentencia, parte por parte.

- `SELECT * FROM contribuyentes_lab01 ORDER BY rut` es la consulta normal.

```
sql
%%sql
-- Celda 4.2
SELECT * FROM contribuyentes_lab01 ORDER BY rut
```

Lo que sale en pantalla.

```
text
rut          razon_social          segmento
76171162-8   Huemul Alimentos EIRL           GRANDE
77746521-K   Pehuen Logistica EIRL           MICRO
77884562-8   Araucaria Ferretería SpA        PEQUENA
78800840-6   Copihue Maquinarias EIRL        MEDIANA
4 filas.
```

Cómo se lee. Araucaria dice PEQUENA, y esta vez sin pedir ninguna versión. Es lo que la tabla dice ahora.

Celda 4.3 · El historial

La pregunta. ¿Se perdió la corrección?

Por qué ahora. Volviste atrás. Falta ver qué quedó registrado.

En el almacén. Es mirar el registro de qué página mandó en cada momento.

La sentencia, parte por parte.

- `made_current_at` es cuándo esa página pasó a mandar, en UTC.
- `snapshot_id` es el número de la página.
- `is_current_ancestor` dice si la página está en el camino que lleva a la que manda hoy.
- `FROM mi_espacio.contribuyentes_lab01.history` es la vista de sistema del historial, con sus tres partes.
- `ORDER BY made_current_at` ordena de lo más viejo a lo más nuevo.

```
sql
%%sql
-- Celda 4.3
SELECT made_current_at, snapshot_id, is_current_ancestor
FROM mi_espacio.contribuyentes_lab01.history
ORDER BY made_current_at
```

Lo que sale en pantalla.

```
text
made_current_at    snapshot_id        is_current_ancestor
2026-09-25 18:21:54.339 3093317371693715250 True
2026-09-25 18:21:57.535 962689764919215466 False
2026-09-25 18:21:59.145 3093317371693715250 True
3 filas.
```

Cómo se lee. Tres momentos. La página `3093317371693715250` mandó desde las 18:21:54.339 UTC. La `962689764919215466` mandó desde las 18:21:57.535 UTC y tiene `False`. La `3093317371693715250` volvió a mandar a las 18:21:59.145 UTC, que es la vuelta atrás.

El `False` dice que la corrección existió pero ya no está en la línea que manda. No se borró. Todavía se puede mirar con `VERSION AS OF 962689764919215466`, o volver a ella.

Preguntas frecuentes

¿De dónde saco mi número de página?

De la celda 2.1, primera fila, columna `snapshot_id`. Es un número largo y es de tu tabla. El de la solución no te sirve, porque cada tabla genera los suyos, y cada vez que creas la tabla de nuevo salen otros.

¿Por qué `VERSION AS OF` lleva el número sin comillas y `TIMESTAMP AS OF` la fecha con comillas?

Porque el número de página es un número y la fecha es un texto que Spark convierte a instante. En `rollback_to_snapshot` pasa lo mismo, la tabla va entre comillas porque es un texto y el número va sin comillas.

¿La hora de `TIMESTAMP AS OF` es la de Chile?

No. Spark trabaja en UTC, igual que `.snapshots`. Si copias la hora tal como sale en la celda 2.1, funciona. Si escribes la hora de Chile, estás tres horas antes y puede que la tabla todavía no existiera.

¿Qué pasa si pido una hora anterior a la primera página?

Spark responde que no encuentra una página anterior a esa fecha. La tabla no existía todavía, y eso también es una respuesta.

¿`rollback_to_snapshot` borra la corrección?

No. Solo cambia qué página manda. La corrección sigue en la libreta, marcada con `False` en el historial, y se puede volver a ella.

¿Puedo mirar el pasado en producción?

Sí. `VERSION AS OF` y `TIMESTAMP AS OF` solo leen. `rollback_to_snapshot` en cambio es una operación de administrador, porque cambia lo que ven todos.

Cierre

La pregunta. ¿Qué te llevas de este laboratorio?

Una tabla Iceberg recuerda. Cada escritura deja una página, y las páginas viejas se pueden leer o volver a declarar vigentes.

En el almacén. El almacenero no usa goma. Hojea hacia atrás para saber qué decía un renglón, y si una corrección estaba mal, mueve el clavo a la página anterior. Ninguna página se arranca.

Lo que hiciste	Con qué	Qué cambió
Ver la lista de páginas	<code>. snapshots</code>	nada
Mirar el pasado por número	<code>VERSION AS OF</code>	nada
Mirar el pasado por fecha	<code>TIMESTAMP AS OF</code>	nada
Volver al pasado	<code>CALL ... rollback_to_snapshot</code>	cuál página manda

Los números de la solución ejecutada.

Momento	Página que manda	Araucaria
Después de la carga	<code>3093317371693715250</code>	PEQUENA
Después de la corrección	<code>962689764919215466</code>	MEDIANA
Después de volver atrás	<code>3093317371693715250</code>	PEQUENA

Mirar atrás no cambia nada. Volver atrás cambia cuál página manda. Ninguna de las dos borra una página.